

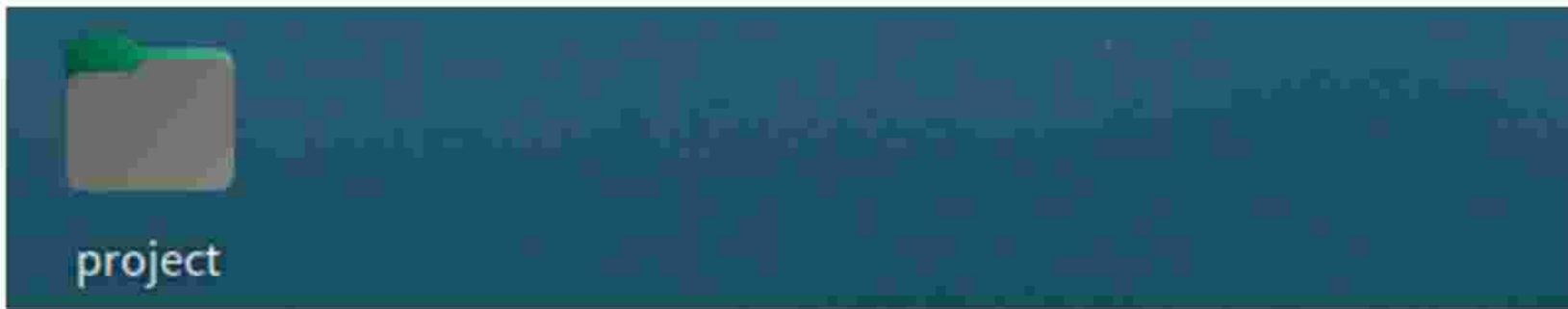


CRUD

CREATE READ UPDATE DELETE
NODE.JS DAN REACT.JS

PERSIAPAN AWAL

Buatlah sebuah direktori yang nantinya digunakan untuk menampung proyek.



Jalankan proyek tersebut dengan menggunakan aplikasi vscode.

```
hk@hk-Linux: ~/Desktop/project
hk@hk-Linux: ~/Desktop/api-users x hk@hk-Linux: ~/Desktop/project x
hk@hk-Linux:~/Desktop$ cd project/
hk@hk-Linux:~/Desktop/project$ code .
```

Buatlah sebuah folder dengan nama backend.

```
✓ PROJECT  
  > backend
```

Masuk kedalam folder backend dan siapkan sebuah package.json

```
● hk@hk-Linux:~/Desktop/project$ cd backend  
● hk@hk-Linux:~/Desktop/project/backend$ npm init -y
```

Install express, mysql, cors, dan nodemon

```
● hk@hk-Linux:~/Desktop/project/backend$ npm install express mysql cors nodemon
```

Tambahkan script "start":"nodemon server.js" kedalam file package.json

```
"scripts": {  
  "test": "echo \"Error: no test specified\" && exit 1",  
  "start": "nodemon server.js"  
},
```

Tambahkan script "type":"module" kedalam file package.json

```
{  
  "name": "backend",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "type": "module",  
}
```

Install dotenv dengan perintah dibawah

```
● hk@hk-Linux:~/Desktop/project/backend$ npm install dotenv
```

Install prisma pada devDependencies dengan perintah dibawah.

```
• hk@hk-Linux:~/Desktop/project/backend$ npm install -D prisma
```

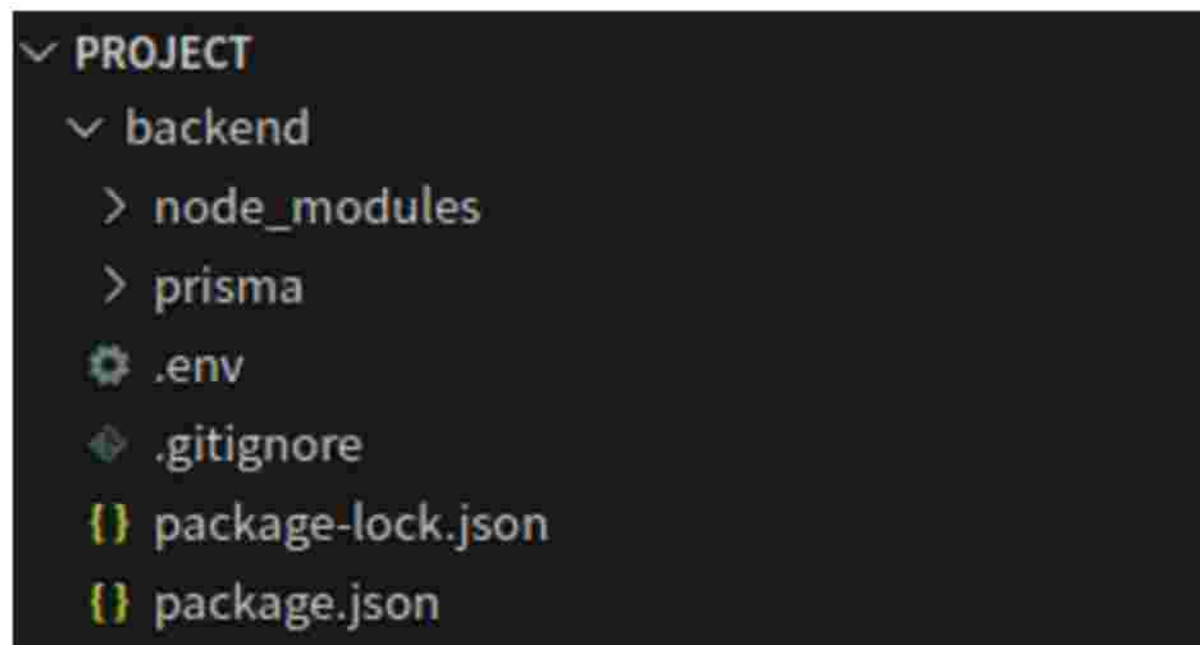
Install prisma client dengan perintah dibawah.

```
• hk@hk-Linux:~/Desktop/project/backend$ npm install @prisma/client
```

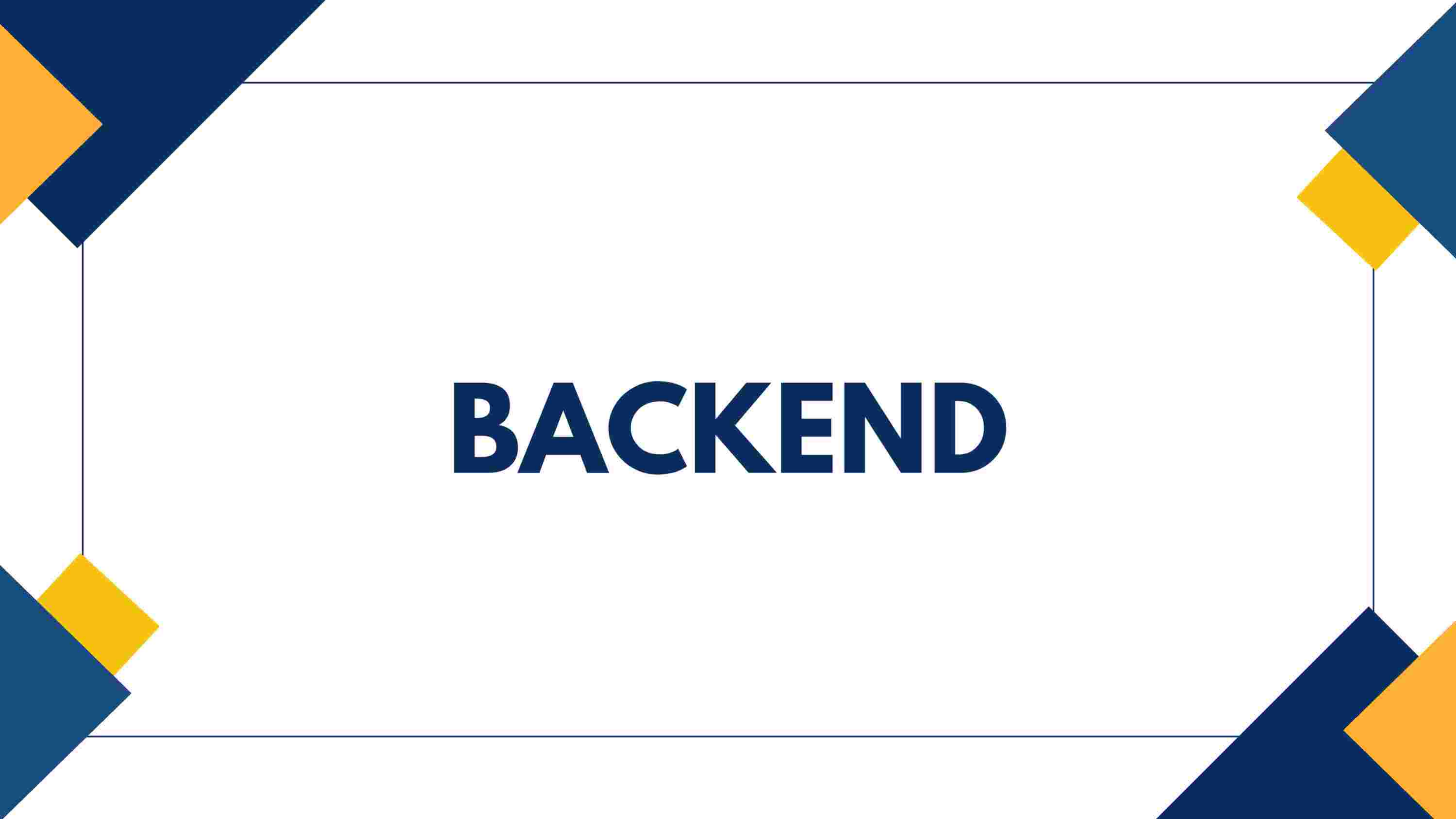
inisialisasi prisma dengan perintah dibawah.

```
• hk@hk-Linux:~/Desktop/project/backend$ npx prisma init
```

Maka kita dapat struktur direktori dan package.json sebagai berikut.



```
1 {
2   "name": "backend",
3   "version": "1.0.0",
4   "description": "",
5   "main": "index.js",
6   "type": "module",
7   "scripts": {
8     "test": "echo \\\"Error: no test specified\\\" && exit 1",
9     "start": "nodemon server.js"
10  },
11  "keywords": [],
12  "author": "",
13  "license": "ISC",
14  "dependencies": {
15    "@prisma/client": "^6.14.0",
16    "cors": "^2.8.5",
17    "dotenv": "^17.2.1",
18    "express": "^5.1.0",
19    "mysql": "^2.18.1",
20    "nodemon": "^3.1.10"
21  },
22  "devDependencies": {
23    "prisma": "^6.14.0"
24  }
25 }
```

A decorative border in the corners of the slide, consisting of overlapping dark blue and yellow geometric shapes (triangles and squares).

BACKEND

Hapus output yang terletak pada schema.prisma dan ubah postgresql dengan mysql.

```
generator client {  
  provider = "prisma-client-js"  
}  
  
datasource db {  
  provider = "mysql"  
  url      = env("DATABASE_URL")  
}
```

konfigurasi database pada file .env dan buat database dengan nama siswa

```
DATABASE_URL="mysql://root:Root*123@localhost:3306/siswa"
```

Buatlah sebuah tabel dengan cara membuat model didalam schema.prisma

```
model datasiswa{  
  id Int @id @default(autoincrement())  
  nama String  
  umur Int  
}
```

Buatlah database dan tabel yang telah disiapkan dengan menggunakan migration dengan perintah berikut.

```
● hk@hk-Linux:~/Desktop/project/backend$ npx prisma migrate dev
```

Periksalah tabel yang telah dibuat didalam database.

```
mysql> show databases;
+-----+
| Database |
+-----+
| db_perpustakaan |
| db_review_movie |
| db_univ |
| information_schema |
| mysql |
| performance_schema |
| siswa |
| sys |
| users_db |
+-----+
9 rows in set (0.22 sec)
```



```
mysql> show tables;
+-----+
| Tables_in_siswa |
+-----+
| _prisma_migrations |
| datasiswa |
+-----+
2 rows in set (0.01 sec)
```

Buatlah sebuah file index.js didalam folder backend, kemudian import express, dotenv, dan cors seperti gambar dibawah.

```
import express from "express"
import cors from "cors"
import dotenv from "dotenv"

dotenv.config()

const app=express()

const PORT=8000
app.listen(PORT,()=>{
  console.log("server berjalan");
})
```

Jalankan server dengan perintah dibawah.

```
hk@hk-Linux:~/Desktop/project/backend$ npm start
```

```
> backend@1.0.0 start
> nodemon server.js

[nodemon] 3.1.10
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js index.js`
[dotenv@17.2.1] injecting env (0) from .env -- tip: 🔒 encrypt with Dotenvx: https://dotenvx.com
server berjalan
```

Tambahkan middleware pada index.js dengan perintah dibawah.

```
app.use(cors())  
app.use(express.json())
```



```
import express from "express"  
import cors from "cors"  
import dotenv from "dotenv"  
  
dotenv.config()  
  
const app=express()  
|  
app.use(cors())  
app.use(express.json())  
  
const PORT=8000  
app.listen(PORT,()=>{  
|   console.log("server berjalan");  
})
```

Buatlah folder controller dan routes didalam backend.

```
✓ backend  
  > controller  
  > node_modules
```

Buatlah file siswaController.js didalam folder controller dan siswaRoute.js didalam folder Routes.

```
✓ controller
  JS siswaController.js
  > node_modules
  > prisma
  ✓ routes
    JS siswaRoute.js
```

Buatlah beberapa function untuk melihat data, mengupdate data, dan menghapus yang nantinya akan di export data pada file siswaController.js

```
✓ export const getSiswa=async(req, res)=>{
  }

✓ export const getSiswaById=async(req, res)=>{
  }

✓ export const createSiswa=async(req, res)=>{
  }
```

```
✓ export const updateSiswa=async(req, res)=>{
  }
  |
✓ export const deleteSiswa=async(req, res)=>{
  }
```

Import express pada file siswaRoute.js sehingga memungkinkan untuk menggunakan express router dengan code dibawah.

```
import express from "express"  
const router = express.Router()  
  
export default router
```

import method yang telah dibuat pada file siswaController.js kedalam file siswaRouter.js menggunakan code dibawah.

```
import express from "express"  
import {getSiswa, getSiswaById, createSiswa, updateSiswa, deleteSiswa} from "../controller/siswaController.js"  
const router = express.Router()
```

Buatlah route dan endpoint pada file siswaRoute.js sesuai dengan method yang telah di import.

```
router.get('/siswa',getSiswa)
router.get('/siswa/:id',getSiswaById)
router.post('/siswa',createSiswa)
router.put('/siswa/:id',updateSiswa)
router.delete('/siswa/:id',deleteSiswa)
```



```
import express from "express"
import {getSiswa,getSiswaById,createSiswa,updateSiswa,deleteSiswa} from "../controller/siswaController.js"
const router =express.Router()

router.get('/siswa',getSiswa)
router.get('/siswa/:id',getSiswaById)
router.post('/siswa',createSiswa)
router.put('/siswa/:id',updateSiswa)
router.delete('/siswa/:id',deleteSiswa)

export default router
```

import siswaRoute kedalam index.js (entry point) menggunakan code dibawah.

```
import express from "express"
import cors from "cors"
import dotenv from "dotenv"
import siswaRoute from "../routes/siswaRoute.js"
```

Tambahkan sebuah middleware siswaRoute kedalam index.js menggunakan code dibawah.

```
app.use(cors())  
app.use(express.json())  
app.use(siswaRoute)
```



```
import express from "express"  
import cors from "cors"  
import dotenv from "dotenv"  
import siswaRoute from "../routes/siswaRoute.js"  
  
dotenv.config()  
  
const app=express()  
  
app.use(cors())  
app.use(express.json())  
app.use(siswaRoute)  
|  
const PORT=8000  
app.listen(PORT,()=>{  
  console.log("server berjalan");  
})
```

Import prisma client kedalam file siswaController.js menggunakan code dibawah.

```
import {PrismaClient} from "@prisma/client"
```

Buatlah sebuah variabel baru dengan nama prisma yang dapat mempermudah penggunaan prisma client.

```
import {PrismaClient} from "@prisma/client"  
const prisma = new PrismaClient
```

Tambahkan async dan try-catch pada setiap method yang telah dibuat.

```
export const getSiswa=async(get, res)=>{  
  try {  
  
  } catch (error) {  
  
  }  
}
```

Buatlah sebuah variabel yang akan digunakan untuk menampilkan data pada method getSiswa.

```
export const getSiswa=async(req, res)=>{  
  try {  
    const reponse = await prisma.siswa.findMany()  
    res.status(200).json(reponse)
```

Tambahkan code dibawah untuk menampilkan pesan jika terjadi error pada method.

```
} catch (error) {  
  res.status(400).json({message:error.message})  
}
```



```
export const getSiswa=async(req, res)=>{  
  try {  
    const reponse = await prisma.siswa.findMany()  
    res.status(200).json(reponse)  
  } catch (error) {  
    res.status(400).json({message:error.message})  
  }  
}
```

Lakukan pengujian menggunakan prisma studio dan postman/thunderclient.

The screenshot shows the Postman interface. On the left, the 'Query' tab is selected, showing the URL 'http://localhost:8000/siswa' and a 'Send' button. Below the URL bar, there are tabs for 'Query', 'Headers', 'Auth', 'Body', 'Tests', and 'Pre Run'. The 'Query' tab is active, and it shows a table for 'Query Parameters' with columns 'parameter' and 'value'. On the right, the 'Response' tab is selected, showing the status '300 Multiple Choices', 'Size: 35 Bytes', and 'Time: 115 ms'. The response body is a JSON array with one object:

```
[  
  {  
    "id": 1,  
    "nama": "haris",  
    "umur": 30  
  }  
]
```

Duplikat code yang terdapat pada method getSiswa kemudian paste kedalam method getSiswaById.

```
export const getSiswaById=async(req, res)=>{  
  try {  
    const reponse = await prisma.siswa.findMany()  
    res.status(200).json(reponse)  
  } catch (error) {  
    res.status(400).json({message:error.message})  
  }  
}
```

Ubahlah fungsi findMany menjadi findUnique() lalu tambahkan opsi where berdasarkan id yang diambil dari parameter yang dimasukkan.

```
try {  
  const reponse = await prisma.siswa.findUnique({  
    where:{  
      id : req.params.id  
    }  
  })  
}
```

Tambahkan Number/parselInt pada code id sehingga mengubah inputan dari string menjadi number

```
where:{  
  id : Number(req.params.id)  
}
```



```
export const getSiswaById=async(req, res)=>{  
  try {  
    const reponse = await prisma.siswa.findUnique({  
      where:{  
        id : Number(req.params.id)  
      }  
    })  
    res.status(200).json(reponse)  
  } catch (error) {  
    res.status(400).json({message:error.message})  
  }  
}
```

Lakukan pengujian terhadap method yang telah dibuat.

The screenshot shows a REST client interface. On the left, the 'Tests' tab is active, showing a 'Scripting New' test. The main area displays a GET request to 'http://localhost:8000/siswa/1' with a 'Send' button. The right panel shows the response details: Status: 300 Multiple Choices, Size: 33 Bytes, Time: 223 ms. The response body is a JSON object: { "id": 1, "nama": "haris", "umut": 30 }.

select	action	value

Duplikat kembali code yang terdapat pada method getSiswa kemudian paste kedalam method createSiswa.

```
export const createSiswa=async(req, res)=>{  
  try {  
    const reponse = await prisma.siswa.findMany()  
    res.status(200).json(reponse)  
  } catch (error) {  
    res.status(400).json({message:error.message})  
  }  
}
```

tambahkan request.body kedalam method createSiswa.

```
export const createSiswa=async(req, res)=>{  
  const {nama,umur}=req.body  
  try {  
    const reponse = await prisma.siswa.findMany()
```

Ubahlah fungsi findMany() menjadi create() lalu masukkan opsi berisi data yang akan dikirimkan kedalam database.

```
try {  
  const reponse = await prisma.siswa.create({  
    data: {  
      nama: nama,  
      umur: umur  
    }  
  })  
}
```



```
export const createSiswa=async(req, res)=>{  
  const {nama,umur}=req.body  
  try {  
    const reponse = await prisma.siswa.create({  
      data: {  
        nama: nama,  
        umur: umur  
      }  
    })  
    res.status(200).json(reponse)  
  } catch (error) {  
    res.status(402).json({message:error.message})  
  }  
}
```

Lakukan pengujian terhadap method createSiswa.

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:8000/siswa/
- Body:** JSON format with content:

```
{  
  "nama": "Haura",  
  "umur": 3  
}
```
- Status:** 200 OK
- Size:** 32 Bytes
- Time:** 109 ms
- Response:** JSON format with content:

```
{  
  "id": 2,  
  "nama": "Haura",  
  "umur": 3  
}
```

Duplikat code yang terdapat pada method createSiswa kemudian paste kedalam method updateSiswa.

```
export const updateSiswa=async(req, res)=>{
  const {nama,umur}=req.body
  try {
    const reponse = await prisma.siswa.create({
      data: {
        nama:nama,
        umur:umur
      }
    })
    res.status(200).json(reponse)
  } catch (error) {
    res.status(402).json({message:error.message})
  }
}
```

Ubahlah fungsi create() menjadi update() lalu tambahkan opsi where berdasarkan id yang diambil dari parameter yang dimasukkan.

```
const reponse = await prisma.siswa.update({
  where:{
    id : Number(req.params.id)
  },
  data: {
    nama:nama,
    umur:umur
  }
})
```

Lakukan pengujian pada method updateSiswa.

The screenshot shows a REST client interface. The method is PUT and the URL is http://localhost:8000/siswa/1. The status is 300 Multiple Choices, size is 31 Bytes, and time is 12 ms. The request body is a JSON object: {"nama": "Robi", "umur": 5}. The response body is a JSON object: {"id": 1, "nama": "Robi", "umur": 5}.

Query	Headers ²	Auth	Body ¹	Tests	Pre Run
JSON Content					
1 { 2 "nama": "Robi", 3 "umur": 5 4 }					

Response	Headers ⁷	Cookies	Results	Docs
1 { 2 "id": 1, 3 "nama": "Robi", 4 "umur": 5 5 }				

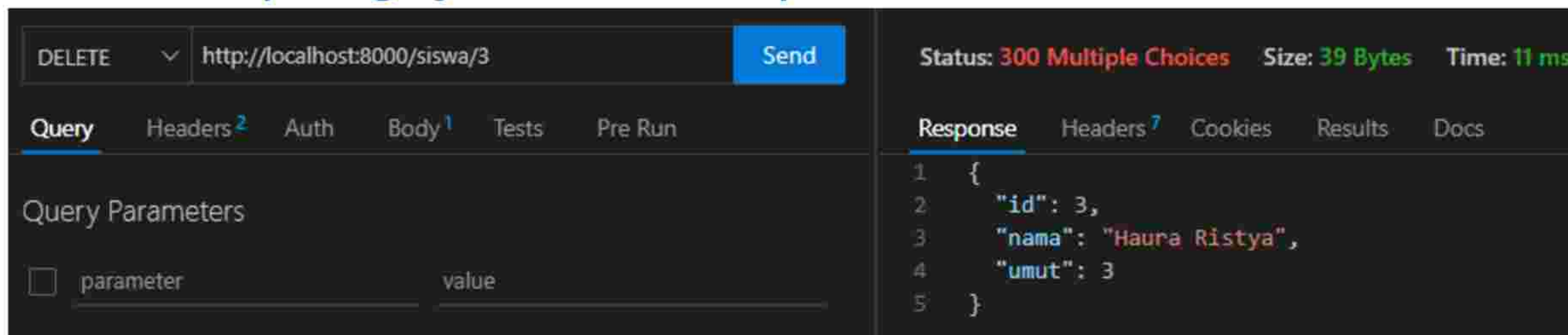
Duplikat code yang terdapat pada method updateSiswa kemudian paste kedalam method deleteSiswa.

```
export const deleteSiswa=async(req, res)=>{
  const {nama,umur}=req.body
  try {
    const reponse = await prisma.siswa.update({
      where:{
        id : Number(req.params.id)
      },
      data: {
        nama:nama,
        umur:umur
      }
    })
    res.status(200).json(reponse)
  } catch (error) {
    res.status(403).json({message:error.message})
  }
}
```

Ubahlah fungsi update() menjadi delete() serta hapus bagian request.body dan opsi data.

```
export const deleteSiswa=async(req, res)=>{
  try {
    const reponse = await prisma.siswa.delete({
      where:{
        id : Number(req.params.id)
      }
    })
    res.status(200).json(reponse)
  } catch (error) {
    res.status(403).json({message:error.message})
  }
}
```

Lakukan pengujian terhadap method deleteSiswa.



The screenshot shows a REST client interface with a DELETE request to `http://localhost:8000/siswa/3`. The response is a 300 Multiple Choices status with a JSON body containing student information.

Method	URL	Status	Size	Time
DELETE	http://localhost:8000/siswa/3	300 Multiple Choices	39 Bytes	11 ms

Tab	Content
Query	Query Parameters table with columns 'parameter' and 'value'.
Response	JSON response: <pre>{ "id": 3, "nama": "Haura Ristya", "umut": 3}</pre>

The image features a white background with a thin blue rectangular border. In each of the four corners, there are overlapping geometric shapes: a dark blue triangle and a yellow square, both tilted at a 45-degree angle. The word "FRONTEND" is centered in the middle of the page in a bold, dark blue, sans-serif font.

FRONTEND

Buka terminal/command prompt baru, kemudian keluar dari folder backend menggunakan perintah `cd ..`. lalu install react beserta direktorinya menggunakan code berikut.

```
D:\hk\project\backend>cd ..  
  
D:\hk\project>npx create-react-app frontend
```

Perhatikan folder/direktori yang ada, jika penginstallan berhasil maka akan tampil seperti berikut.

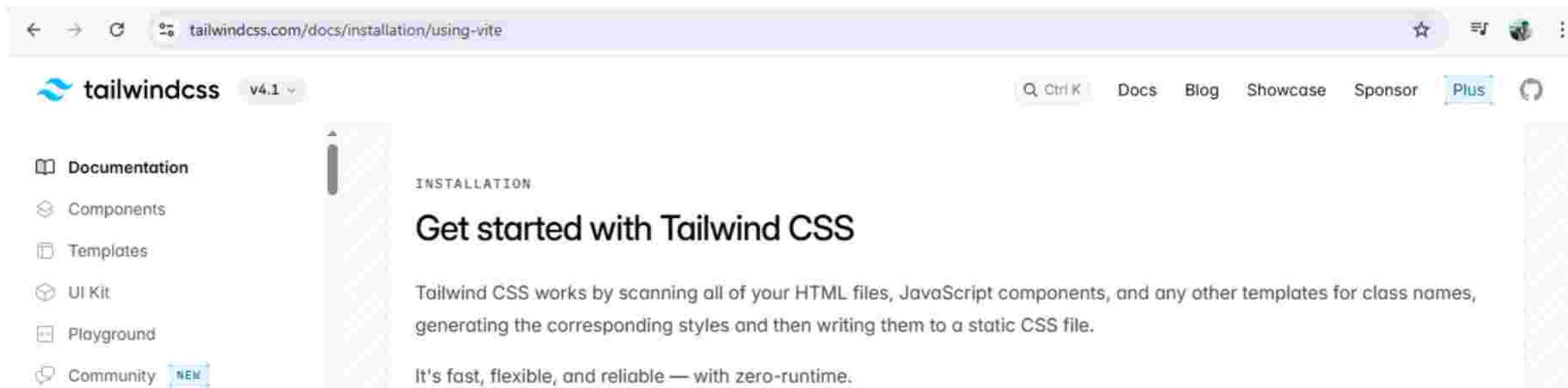
```
▼ PROJECT  
  > backend  
  ▼ frontend  
    > node_modules  
    > public  
    > src  
    .gitignore  
    {} package-lock.json  
    {} package.json  
    ⓘ README.md
```

Masuk kedalam folder frontend, kemudian install axios (karena akan berhubungan dengan api) dan react router dom (karena akan menggunakan router) dan install swr (karena akan menggunakan realtime data fetching).

```
D:\hk\project>cd frontend
```

```
D:\hk\project\frontend>npm install axios react-router-dom swr
```

Lakukan instalasi dengan cara mengikuti guide yang terdapat pada [website resmi tailwind](https://tailwindcss.com/docs/installation/using-vite).



Hapus import logo from './logo.svg' dan ./App.css yang terletak pada file App.js untuk menghilangkan warning.

```
import logo from './logo.svg';  
import './App.css';
```

Hapus file yang tidak diperlukan (App.css, App.test.js, logo.svg, reportWebVitals.js, setupTests.js)

```
# App.css  
JS App.js  
JS App.test.js  
# index.css  
JS index.js  
📁 logo.svg  
JS reportWebVitals.js  
JS setupTests.js
```



```
JS App.js  
# index.css  
JS index.js
```

Hapus reportWebVitals dan function reportWebVitals yang terdapat pada file index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App.js';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Buatlah sebuah folder baru bernama components didalam folder src kemudian buat sebuah file dengan nama daftasSiswa.jsx

```
▼ src
  ▼ components
    🛠 daftasSiswa.jsx
  JS App.js
  # index.css
  JS index.js
```

Generate komponen react dengan menggunakan perintah rafce (jika sudah menginstall plugin) atau ketik code seperti berikut.

```
import React from 'react'

const daftarSiswa = () => {
  return (
    <div>daftarSiswa</div>
  )
}

export default daftarSiswa
```

Bukalah file daftarSiswa.jsx kemudian siapkan sebuah tabel untuk menampung data yang terdapat

```
const daftarSiswa = () => {
  return (
    <div>
      <div>daftarSiswa</div>
      <div>
        <table>
          <thead>
            <tr>
              <th>No</th>
              <th>Nama</th>
              <th>Umur</th>
              <th>Aksi</th>
            </tr>
          </thead>
```

Bukalah file App.js kemudian import react touter dom seperti code dibawah

```
import {BrowserRouter, Routes, Route} from "react-router-dom"
```

Hapus bagian <h1> kemudian ganti dengan BrowserRouter, Routes, dan Route.

```
return (  
  <div>  
    <BrowserRouter>  
      <Routes>  
        <Route/>  
      </Routes>  
    </BrowserRouter>  
  </div>  
>;
```

Tambahkan atribut path "/" yang artinya itu home kemudian tambahkan element untuk merender komponen.

```
<BrowserRouter>  
  <Routes>  
    <Route path="/" element={}>/>  
  </Routes>  
</BrowserRouter>
```

import daftarSiswa kedalam file App.js kemudian masukkan daftarSiswa kedalam element yang telah disiapkan.

```
import {BrowserRouter, Routes, Route} from "react-router-dom"
import DaftarSiswa from "../components/daftarSiswa.jsx";
```

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<DaftarSiswa/>}/>
  </Routes>
</BrowserRouter>
```



```
import {BrowserRouter, Routes, Route} from "react-router-dom"
import DaftarSiswa from "../components/daftarSiswa.jsx";

function App() {
  return (
    <div>
      <BrowserRouter>
        <Routes>
          <Route path="/" element={<DaftarSiswa/>}/>
        </Routes>
      </BrowserRouter>
    </div>
  );
}

export default App;
```

Import link dari react router dom kedalam file daftarSiswa.jsx

```
import React from "react"
import { Link } from "react-router-dom"
```

Import axios yang digunakan untuk fetching data dan Import SWR karena fetching data akan dilakukan secara realtime pada file daftarSiswa.jsx

```
import axios from "axios"  
import useSWR from "swr"
```

Buatlah sebuah method untuk fetching data pada file daftarSiswa.jsx dan masukkan endpoint.

```
const fetcher=async()=>{  
  const response=await axios.get('http://localhost:8000/siswa',{  
    headers:{Accept:'application/json'}})  
  return response.data  
}
```

Tambahkan code dibawah ini untuk menggunakan SWR

```
const DaftarSiswa = () => {  
  const {data, error}=useSWR('http://localhost:8000/siswa',fetcher)  
  if(error) return <h1>Gagal memuat data</h1>;  
  if(!data) return <h1>Loading...</h1>;
```

Lakukan pengulangan pada tabel untuk mendapatkan data dari database.

```
<tbody>  
  {data && data.map((siswa, idx) => (  
  
  )}}
```

Pindahkan seluruh kode pada row tbody kedalam pengulangan yang telah dibuat.

```
<tbody>  
  {data && data.map((siswa, idx) => (  
    <tr key={siswa.id}>  
      <td>{idx+1}</td>  
      <td>{siswa.nama}</td>  
      <td>{siswa.umut}</td>  
      <td></td>  
    </tr>  
  )}}  
</tbody>
```

Pastikan User Interface menampilkan data yang sesuai dengan database.

localhost:3000

Daftar Siswa

No	Nama	Umur	Aksi
1	A	0	
2	B	1	

```
Prompt Perintah - mysql -u root -p
mysql> select * from siswa;
+----+-----+-----+
| id | nama | umur |
+----+-----+-----+
| 1  | A    | 0    |
| 2  | B    | 1    |
+----+-----+-----+
2 rows in set (0.00 sec)

mysql>
```

Tambahkan tailwind css (dapat menggunakan chatGPT) untuk memperindah tampilan tabel.

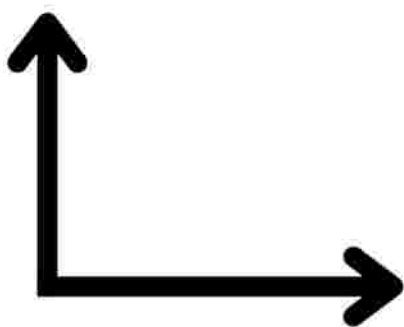
No	Nama	Umur	Aksi
1	A	0	
2	B	1	

Tambahkan sebuah tombol diatas tabel yang nantinya akan digunakan untuk menambahkan data kedalam database.

```
<div className="">
|   <Link to='/tambah' className=''>Tambah Data Siswa</Link>
</div>
```

Tambahkan className="flex justify-end mb-4" pada div tombol yang dibuat agar tombol berada disebalah kanan, dan tambahkan tailwind css untuk tombol tambah daftar siswa.

```
<div className="flex justify-end mb-4">
|   <Link to='/tambah' className='inline-flex items-center gap-2 display-block bg-green-500 text-white'>
</div>
```



Daftar Siswa			
No	Nama	Umur	Aksi
1	A	0	
2	B	1	

Tambahkan tombol edit dan tombol hapus pada kolom aksi.

```
<td className="py-2 px-4 space-x-2">
  <Link to={` /edit/${siswa.id}`} className="">
    Edit
  </Link>
  <button className="">
    Hapus
  </button>
</td>
```

Tambahkan Tailwind css untuk memperindah tombol edit dan hapus.

Daftar Siswa

Tambah Data Siswa

No	Nama	Umur	Aksi	
1	A	0	Edit	Hapus
2	B	1	Edit	Hapus

Tambahkan method onClick pada tombol hapus yang membawa parametel id siswa.

```
<button onClick={() => deleteSiswa(siswa.id)}  
  Hapus  
</button>
```

buatlah method deleteSiswa didalam function DaftarSiswa yang akan digunakan oleh tombol hapus.

```
const deleteSiswa=async(id)=>{  
  if(window.confirm("Apakah Anda yakin ingin menghapus data ini?")){  
    await axios.delete(`http://localhost:8000/siswa/${id}`)  
    mutate(`http://localhost:8000/siswa`)  
  }  
}
```

Tambahkan {useSWRConfig} didalam perintah import swr

```
import React from 'react'  
import axios from 'axios'  
import useSWR,{useSWRConfig} from 'swr'  
import {Link} from 'react-router-dom'
```

Tambahkan `const {mutate}=useSWRConfig()` tepat dibawah fungsi `DaftarSiswa`.

```
const DaftarSiswa = () => {  
  const {mutate}=useSWRConfig()  
  const {data, error}=useSWR('http://localhost:8000/siswa',fetcher)  
  if(error) return <h1>Gagal memuat data</h1>;  
  if(!data) return <h1>Loading...</h1>;  
}
```

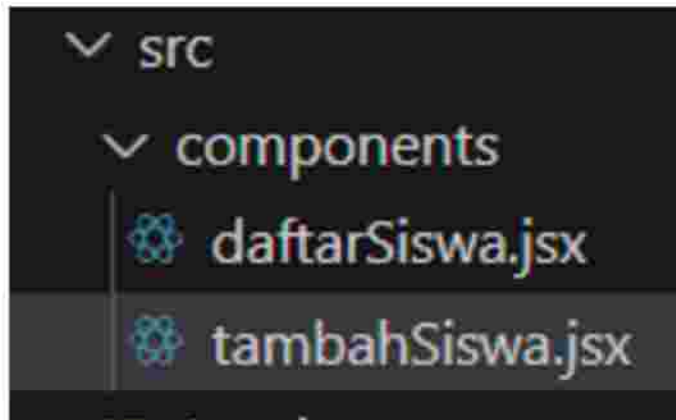
Cobalah untuk menekan tombol hapus pada UI untuk memastikan tombol berjalan dengan normal.

Daftar Siswa

Tambah Data Siswa

No	Nama	Umur	Aksi
1	B	1	EditHapus

Buatlah sebuah file tambahSiswa.jsx pada folder component.



Ketik rafce untuk membentuk struktur react secara otomatis pada file tambahSiswa.jsx

```
import React from 'react'

const tambahSiswa = () => {
  return (
    <div>tambahSiswa</div>
  )
}

export default tambahSiswa
```

Buatlah sebuah form yang akan digunakan untuk menambah data pada tambahSiswa.jsx.

```
const tambahSiswa = () => {  
  return (  
    <div>  
      <form>  
        <div>  
          <label>Nama:</label>  
          <input type="text" />  
        </div>  
        <div>  
          <label>Umur:</label>  
          <input type="text" />  
        </div>  
        <div>  
          <button type="submit">Simpan</button>  
        </div>  
      </form>  
    </div>  
  )  
}
```

Masuklah ke folder App.js lalu import komponen TambahSiswa yang mengacu pada file tambahSiwa.jsx

```
import {BrowserRouter, Routes, Route} from "react-router-dom"  
import DaftarSiswa from "../components/daftarSiswa.jsx";  
import TambahSiswa from "../components/tambahSiswa.jsx";
```

Tambahkan Route path dengan endpoint /tambah dengan komponen TambahSiswa.

```
<Routes>  
  <Route path="/" element={<DaftarSiswa/>}/>  
  <Route path="/tambah" element={<TambahSiswa/>}/>  
</Routes>
```

Masuklah ke browser, lalu cobalah menekan tombol tambah data siswa, jika berhasil maka akan diarahkan ke halaman tambah siswa.



Gunakan bantuan Tailwind css untuk memperindah tampilan tambah siswa yang telah dibuat.



The screenshot shows a web form titled "Tambah Siswa". It contains two input fields: "Nama:" with a placeholder "Masukkan nama siswa" and "Umur:" with a placeholder "Masukkan umur siswa". A blue "Simpan" button is located at the bottom right of the form.

Import useState, axios, dan useNavigate di bagian paling atas pada file tambahSiswa.jsx

```
import React,{useState} from 'react'  
import axios from 'axios'  
import { useNavigate } from 'react-router-dom'
```

Tambahkan beberapa state didalam tambahSiswa yang nantinya digunakan untuk menyimpan inputan dari UI.

```
const tambahSiswa = () => {  
  const [nama, setNama]=useState('')  
  const [umur, setUmur]=useState('')  
  const navigate=useNavigate()  
}
```

Tambahkan value didalam input nama pada form yang telah dibuat serta menambahkan onChange event dengan target state yang telah dibuat.

```
value={nama} onChange={(e) => setNama(e.target.value)}/>
```

Tambahkan juga value didalam input umur pada form yang telah dibuat serta menambahkan onChange event dengan target state yang telah dibuat.

```
value={umur} onChange={(e) => setUmur(e.target.value)}/>
```

Tambahkan onSubmit pada tag form serta menambahkan method simpanSiswa didalamnya.

```
<form onSubmit={simpanSiswa}>
```

Tambahkan fungsi simpanSiswa diatas return.

```
const simpanSiswa=async(e)=>{
  e.preventDefault()
  await axios.post('http://localhost:8000/siswa',{
    nama:nama,
    umur:Number(umur)
  })
  navigate('/')
}
```

Ubahlah fungsi tambahSiswa menjadi TambahSiswa agar tidak terjadi error

```
const tambahSiswa = () => {
  const [nama,setNama]=useState('')
  const [umur,setUmur]=useState('')
  const navigate=useNavigate()
```



```
const TambahSiswa = () => {
  const [nama,setNama]=useState('')
  const [umur,setUmur]=useState('')
  const navigate=useNavigate()
```

Lakukan pengujian dengan cara memasukkan data siswa yang baru untuk memastikan tombol sudah berjalan dengan normal.

Tambah Siswa

Nama:

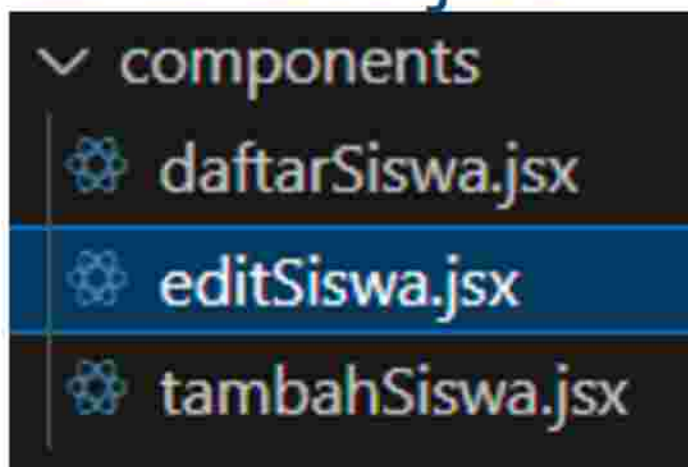
Umur:



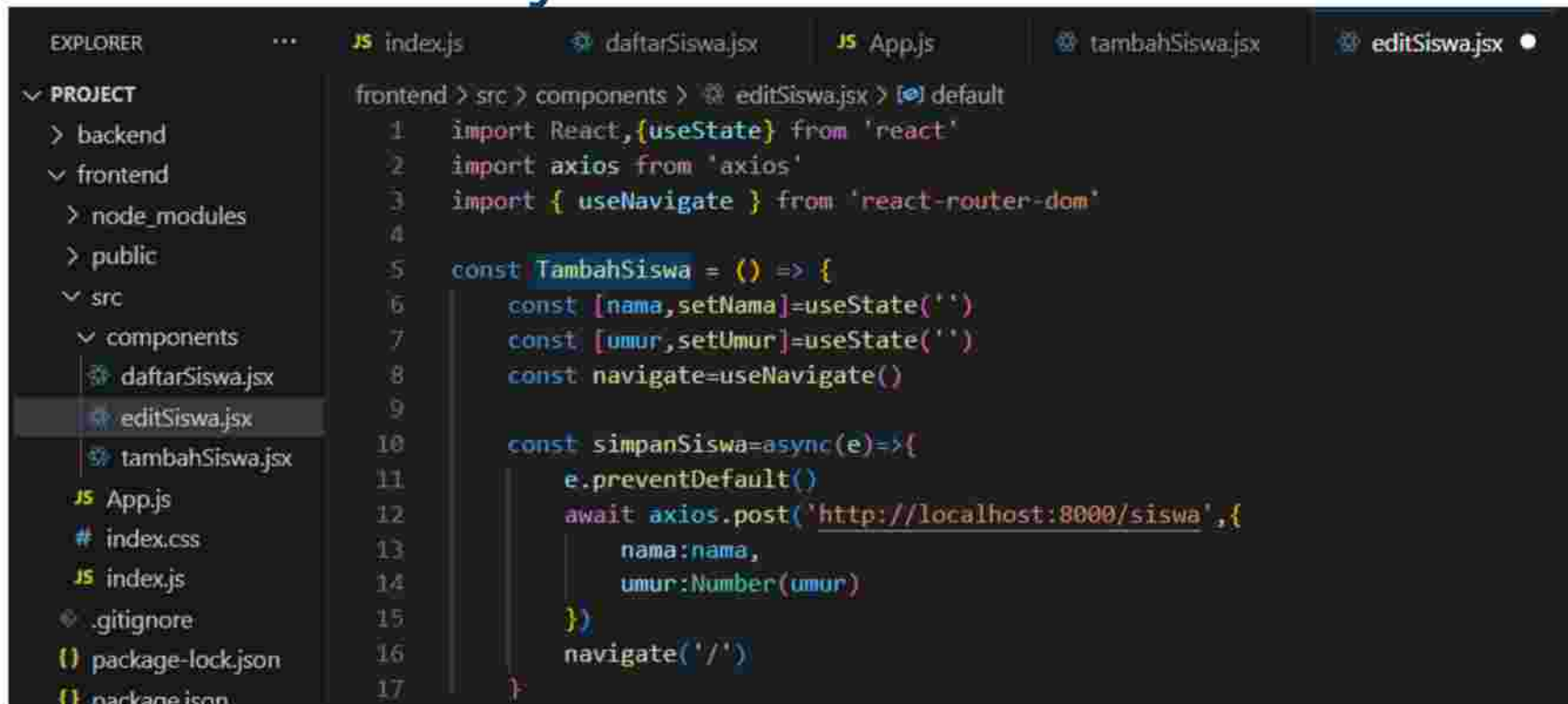
Daftar Siswa

No	Nama	Umur	Aksi
1	B	1	Edit Hapus
2	C	2	Edit Hapus
3	D	100	Edit Hapus

Buatlah satu file baru didalam folder components dengan nama editSiswa.jsx.



Copy keseluruhan code yang terletak didalam file tambahSiswa.jsx kedalam file editSiswa.jsx

A screenshot of the Visual Studio Code editor. The Explorer panel on the left shows a project structure with folders 'backend' and 'frontend', and a file 'editSiswa.jsx' selected under 'src > components'. The main editor area shows the code for 'editSiswa.jsx'. The code includes imports for React, axios, and useNavigate from 'react-router-dom'. It defines a function 'TambahSiswa' which uses useState for 'nama' and 'umur', and useNavigate for navigation. The function 'TambahSiswa' calls axios.post to save student data to 'http://localhost:8000/siswa'.

```
1 import React,{useState} from 'react'
2 import axios from 'axios'
3 import { useNavigate } from 'react-router-dom'
4
5 const TambahSiswa = () => {
6   const [nama,setNama]=useState('')
7   const [umur,setUmur]=useState('')
8   const navigate=useNavigate()
9
10  const simpanSiswa=async(e)=>{
11    e.preventDefault()
12    await axios.post('http://localhost:8000/siswa',{
13      nama:nama,
14      umur:Number(umur)
15    })
16    navigate('/')
17  }
```

Ubahlah function TambahSiswa menjadi EditSiswa, kemudian jangan lupa mengubah export default yang terletak dibaris paling bawah menjadi EditSiswa.

```
const EditSiswa = () => {
```

```
export default EditSiswa
```

Ubahlah button simpan yang terletak didalam file editSiswa.jsx menjadi button update.

```
bold">Simpan</button> → ld">Update</button>
```

Ubahlah method simpanSiswa menjadi update siswa, kemudian ubah post menjadi put/patch.

```
const simpanSiswa=async(e)=>{  
  e.preventDefault()  
  await axios.post('http://localhost:8000/siswa',{  
    nama:nama,  
    umur:Number(umur)  
  })  
  navigate('/')  
}
```



```
const updateSiswa=async(e)=>{  
  e.preventDefault()  
  await axios.put('http://localhost:8000/siswa',{  
    nama:nama,  
    umur:Number(umur)  
  })  
  navigate('/')  
}
```

Ubahlah onSubmit yang terletak pada tag form menjadi updateSiswa.

`<form onSubmit={simpanSiswa}>` → `<form onSubmit={updateSiswa}>`

Masuk kedalam file App.js lalu import component editSiswa.

```
import {BrowserRouter, Routes, Route} from "react-router-dom";  
import DaftarSiswa from "../components/daftarSiswa.jsx";  
import TambahSiswa from "../components/tambahSiswa.jsx";  
import EditSiswa from "../components/editSiswa.jsx";
```

Buatlah Route path baru yang berisi endpoint (jangan lupa menambahkan parameter id), kemudian masukkan komponen EditSiswa didalam element.

```
<Routes>  
  <Route path="/" element={<DaftarSiswa/>}/>  
  <Route path="/tambah" element={<TambahSiswa/>}/>  
  <Route path="/edit/:id" element={<EditSiswa/>}/>  
</Routes>
```

Cobalah buka browser kemudian tekan tombol edit. Jika saat ditekan dan menampilkan halaman edit, maka tombol sudah berjalan dengan normal.

Daftar Siswa			
			Tambah Data Siswa
No	Nama	Umur	Aksi
1	B	1	Edit Hapus
2	C	2	Edit Hapus
3	D	100	Edit Hapus



Edit Siswa	
Nama:	Masukkan nama siswa
Umur:	Masukkan umur siswa
Simpan	

Masuk kembali kedalam file editSiswa.jsx, kemudian import useEffect didalam react, yang nantinya akan digunakan untuk menampilkan text didalam form edit sesuai dengan tombol edit yang ditekan.

```
import React, {useState, useEffect} from 'react'
import axios from 'axios'
import { useNavigate } from 'react-router-dom'
```

Buatlah fungsi useEffect didalam file editSiswa.jsx, kemudian buat method get data by id didalam fungsi tersebut.

```
useEffect(()=>{
  const getSiswaById=async()=>{
    const response=await axios.get(`http://localhost:8000/siswa/${id}`)
    setNama(response.data.nama)
    setUmur(response.data.umur)
  }
  getSiswaById()
},[id])
```

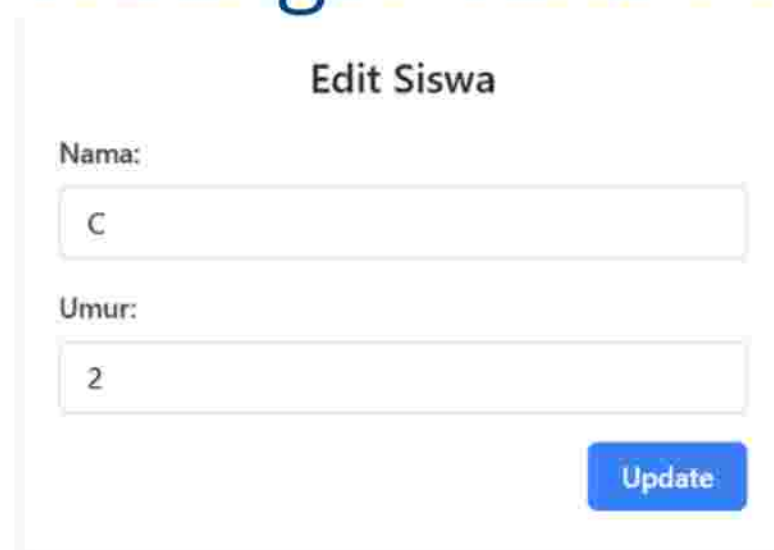
Import useParams pada react-router-dom yang akan digunakan untuk mengambil parameter id.

```
import React,{useState, useEffect} from 'react'  
import axios from 'axios'  
import { useNavigate, useParams } from 'react-router-dom'
```

Tambahkan const {id}=useParams() agar id dapat digunakan didalam method getSiswaById.

```
const EditSiswa = () => {  
  const [nama,setNama]=useState('')  
  const [umur,setUmur]=useState('')  
  const navigate=useNavigate()  
  const {id}=useParams()
```

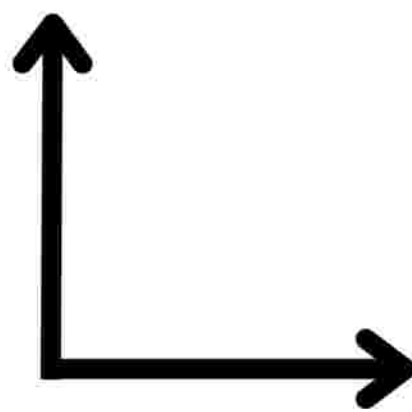
Cobalah buka browser kemudian pastikan kolom edit siswa yang awalnya kosong akan terisi dengan sendirinya.



The screenshot shows a web form titled "Edit Siswa". It contains two input fields: "Nama:" with the value "C" and "Umur:" with the value "2". Below the inputs is a blue "Update" button.

Untuk menjalankan tombol update, maka kita harus mengubah simbol kutip satu (') pada method updateSiswa dengan simbol backtick (` ) kemudian masukkan parameter `${id}`.

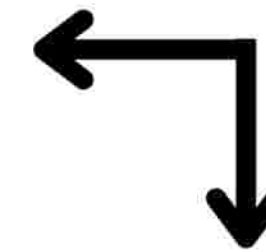
```
const updateSiswa=async(e)=>{  
  e.preventDefault()  
  await axios.put('http://localhost:8000/siswa',{  
    nama:nama,  
    umur:Number(umur)  
  })  
  navigate('/')  
}
```



```
const updateSiswa=async(e)=>{  
  e.preventDefault()  
  await axios.put(`http://localhost:8000/siswa/${id}`, {  
    nama:nama,  
    umur:Number(umur)  
  })  
  navigate('/')  
}
```

Lakukan pengujian terhadap tombol edit, dan pastikan semua perintah berjalan dengan normal.

Daftar Siswa				
				Tambah Data Siswa
No	Nama	Umur	Aksi	
1	B	1	Edit	Hapus
2	Cendana	2	Edit	Hapus
3	D	100	Edit	Hapus

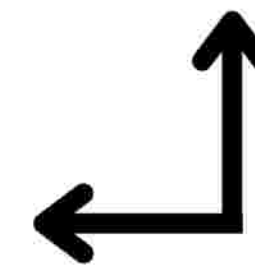


Edit Siswa

Nama:

Umur:

Update



Daftar Siswa				
				Tambah Data Siswa
No	Nama	Umur	Aksi	
1	B	1	Edit	Hapus
2	Cendana	2	Edit	Hapus
3	Dermawan	200	Edit	Hapus



THANK YOU